SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

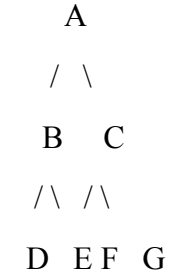
Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
   1 2 3
   4 5 6
   7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion
1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

SIMATS ENGINEERING
CSA17 – ARTIFICIAL INTELLIGENCE
ASSESSMENT TOOL 3: DRY RUN EVALUATION – ANSWERS

Name: ARAVINDH R Reg. No.:192312024

1. Breadth-First Search (BFS)

Given tree: A has children B and C; B has children D and E; C has children F and G. Assume children are visited from left to right. BFS uses a FIFO queue: remove the front node, visit it, and add its unvisited children to the rear.

Iteration	Queue before expansion	Visited node	Queue after expansion
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

BFS traversal order

A → B → C → D → E → F → G

BFS visits nodes level by level. The queue is empty after all seven nodes have been visited.

2. Depth-First Search (DFS)

Assume left-to-right traversal. DFS follows one branch as deeply as possible before backtracking. The recursive/stack traversal is:

Step	Action	Visited order so far
1	Visit A; go to B	A
2	Visit B; go to D	A → B
3	Visit D; no children, backtrack	A → B → D
4	Visit E; no children, backtrack	A → B → D → E
5	Backtrack to A; go to C	A → B → D → E → C
6	Visit F; no children, backtrack	A → B → D → E → C → F
7	Visit G	A → B → D → E → C → F → G

DFS traversal order

A → B → D → E → C → F → G

If using an explicit stack and pushing children onto it, push the right child before the left child so that the left child is popped and visited first.

3. 8-Puzzle: Breadth-First Search

Initial and goal states

Initial state:

1 3 6
5 0 2
4 7 8

Goal state:

1 2 3
4 5 6
7 8 0

The blank (0) moves one square at a time. For the dry-run table, use the conventional move order Up, Down, Left, Right and avoid revisiting states already discovered. Because the goal is at depth 8, it is not reached in the first five expansions; the table below shows the first five queue expansions. Queue labels use compact state notation such as [136/502/478] for rows 136, 502, 478.

Iteration	Current state expanded	Queue before expansion (front first)	Queue after expansion (front first)
1	136/502/478	136/502/478	106/532/478, 136/572/408, 136/052/478, 136/520/478
2	106/532/478	106/532/478, 136/572/408, 136/052/478, 136/520/478	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478
3	136/572/408	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480
4	136/052/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078
5	136/520/478	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078	016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470

Note: Queue entries are written as three rows separated by slashes. To keep the dry-run readable, the queue is shown using compact state notation; entries are in FIFO order.

Expanded nodes and generated states

The first five expanded states are shown in the table. The total number of states generated depends on whether the count means unique discovered states or every successor considered. In the standard BFS implementation with a visited set, the goal is reached after 8 moves from the initial state. A complete search may discover additional states at shallower depths before the goal is dequeued.

Shortest solution path

Step	State (rows separated by /)	Blank move
Initial	1 3 6 / 5 0 2 / 4 7 8	—
1	1 3 6 / 5 2 0 / 4 7 8	Right
2	1 3 0 / 5 2 6 / 4 7 8	Up
3	1 0 3 / 5 2 6 / 4 7 8	Left
4	1 2 3 / 5 0 6 / 4 7 8	Down
5	1 2 3 / 0 5 6 / 4 7 8	Left
6	1 2 3 / 4 5 6 / 0 7 8	Down
7	1 2 3 / 4 5 6 / 7 0 8	Right
Goal	1 2 3 / 4 5 6 / 7 8 0	Right

Why BFS finds the shortest solution

BFS expands states in increasing order of depth: first all states one move away, then all states two moves away, and so on. In an unweighted 8-puzzle, each legal move has equal cost (one move). Therefore, the first time BFS reaches the goal, it has found a path with the minimum possible number of moves, provided repeated states are handled correctly.

Important note on state counts

The assessment asks for the total number of states generated but does not define whether to count repeated successor attempts or only unique states, and its table provides only five rows even though the goal is eight moves away. For a reproducible count, state the implementation's counting rule. The solution path above is an 8-move shortest path; do not treat the five-row table as the complete search.